

Intro to MCP Servers

Presented by Pranav



x



sase

Install now

SETUP · README.md

Windows

`setup.ps1 -CoreOnly`

Mac

`bash setup.sh --core-only`

Pair if blocked



QR: event check-in



Today's workshop

8 min setup · 10 theory · 7 demo · 15 practice
7 security · 5 enterprise · 3 exit · 5 spare

UNDERSTAND

Connect an AI Application to useful software capabilities.

TRACE

Follow a request from the host to the music Application.

IMPLEMENT + TEST

Extend the supplied server with one validated tool.

What MCP is

Model Context Protocol

An open standard that connects AI Applications to tools, data, and reusable instructions.

An MCP server is the program that exposes those capabilities.



Today: extend an existing server using the Python SDK.

Why MCP instead of one prompt?

Workflow step	What MCP adds
One-shot prompt	Gives instructions; tools still need a connection.
Live capabilities	The host discovers tools and input schemas.
Act + observe	Call an action, read state, then adjust.
Reusable workflow	Compatible hosts reuse the same tool interface.



Me when the next
compatible AI host
uses the same server.

MCP does not guarantee correctness. Test tools and verify outcomes.

Host, client, server: the apps you know

HOST · the AI Application you use

Codex · Claude Code · Antigravity
Our demo: `gemini_host.py` uses Gemini
The host coordinates model + tools + permissions.

CLIENT · the MCP connection inside it

Our demo: Python `ClientSession`
Discovers tools, sends calls, receives results.



MCP

SERVER · exposes tools

Our demo: `mcp_server_sdk.py`
Built with FastMCP
Starter: `get_state` + `set_tempo`; you add `set_swing`

Model ≠ host: Gemini/Claude/GPT supply model reasoning; the host Application coordinates the work.
Backend ≠ MCP server: `app.py` changes the music; our server exposes its controls.

Where an API fits

AI HOST

Contains the MCP client

MCP SERVER

Exposes a useful tool

EXISTING API

Applies Application rules

DATA / ACTION

Reads or changes the app

MCP request →

API call →

Read / write →

An MCP tool can call an EXISTING API, Python function, or supported Application command.

1 / 3 · PRIZE QUESTION · Hands up!

Who chooses which MCP tool to call?

A The host B The server C JSON

12 AM. Your team's MCP power tools.

Connect existing servers. Spend your time on the idea.



Figma

DESIGN

Bring UI designs into your coding workflow.



GitHub

BUILD

Inspect code, track issues and work with PRs.



Supabase

DATA

Inspect schemas, query data and manage migrations.



Playwright

TEST

Drive the browser and verify the demo flow.



Vercel

SHIP

Manage deployments and inspect deployment logs.



Sentry

DEBUG

Investigate errors before the judges find them.



Stripe

PAYMENTS

Explore APIs and build a sandbox payment demo.



Context7 · Upstash

DOCS

Pull current library docs and code examples.

Design → build → connect data → test → deploy → fix

Pick a compatible host → add a server → authorize → inspect tools → use it

Start with the 2–3 servers your demo needs. Access and features depend on permissions and plans.

Tools, resources, and prompts

TOOLS

Callable operations
Example: `set_tempo` or `get_state`



RESOURCES

Read-only context and data
Our app: GeneralUser GS
instrument catalog



PROMPTS

Reusable instructions
Example: a review
workflow



Discover → select → invoke

1 · DISCOVER

The client lists tools with their names, descriptions, and input schemas.

2 · SELECT

The model proposes a tool and arguments; the host coordinates the call.

3 · INVOKE

The client calls the tool. The server validates, executes, and returns a result.

`set_tempo` accepts `bpm`, an integer.
Python validation enforces the allowed range.

Ways to build or connect MCP

Option	When to choose it
SDK FASTMCP	Official Python SDK v1; used in this workshop.
Standalone FastMCP	Separate Python package: <code>from fastmcp import FastMCP</code> .
Other SDKs	Official SDKs for your stack, including TypeScript.
Zapier MCP	Managed app actions via Zapier MCP; a Zap is trigger → actions.

DATABUTTON · adjacent app builder

An app builder is not itself an MCP framework.

Choose by tools exposed, host support, auth and deployment.

Links and readiness

github.com/pranavsaigandikota/MCPJAM

aistudio.google.com/api-keys

- 1 Open README.md for setup and launch instructions.
- 2 Start the music Application.
- 3 Open mcp_server_sdk.py.
- 4 Use workshop_client.py for explicit tool tests.

Gemini key: AI demo only. Explicit tool tests need no key.

Demo: an AI controls the music app

“Set the tempo to 120 BPM.”

- 1 Inspect the selected tool and arguments.
- 2 Observe the change in the music Application.
- 3 Call `get_state` to verify the result.



MCP ACTIVITY · trace the call

Host + MCP client

gemini_host.py or the explicit workshop_client.py

↓ Client ↔ server: MCP over stdio

mcp_server_sdk.py

Tool definitions and the Python SDK

↓ Tool function → adapter: Python function call

call_daw()

Backend adapter

↓ Adapter ↔ app: newline-delimited JSON over the local socket

app.py

Music Application · UI and audio

Activity map: where we edit

File	Workshop step
README.md	SETUP · slide 12: launch and test commands
mcp_server_sdk.py	BUILD · slides 20–21: add set_swing here
workshop_client.py	TEST · slides 22–23: discover, call, verify
mcp_server_solution.py	REFERENCE · compare after your attempt

MCP slides 14 + 16 explain the path and contract. app.py is the backend.

MCP ACTIVITY · understand the contract

```
@mcp.tool()  
def set_tempo(bpm: int) -> str:  
    """Set tempo from 40 to 240 BPM."""
```

DECORATOR

Registers the tool

TYPE ANNOTATION

Describes the input

DOCSTRING

Describes its purpose

A useful tool can act and observe

1 · READ STATE

Observe the starting value.

2 · PERFORM AN ACTION

Call a specific tool.

3 · READ STATE AGAIN

Verify the expected result.

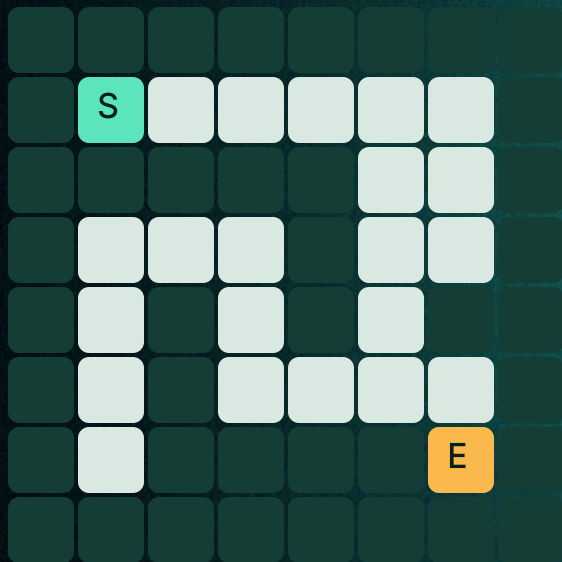
Trust “queued”
or verify the actual state?



Choose the state check.

Can an AI escape a maze using tools?

The model chooses a move. The MCP server exposes controls. The maze app applies it.



S = start E = exit Dark cells = walls

EXPOSED MCP TOOLS

```
move_up()    move_down()  
move_left()  move_right()  
get_maze_state()
```

THE FEEDBACK LOOP

- 1 Read the maze + current position
- 2 Model selects a movement tool
- 3 App returns moved / blocked + position
- 4 Observe again; stop when at_exit = true

WORKSHOP CONNECTION

```
Move tool ↔ set_swing  
get_maze_state ↔ get_state  
Maze app ↔ app.py
```

MCP is the interface. The model plans. The Application enforces valid moves.

Is this different from a “normal MCP”?

Same MCP protocol. Our custom server exposes music capabilities.

CODING AGENT · builds and tests

- Shell, browser or API tools
- Can test an app without MCP
- Prompt alone adds no capabilities
- Verify with actual evidence

YOUR APP + MCP · reusable controls

- Expose focused tools + schemas
- Reuse across compatible hosts
- Production: validate + authorize
- Read state to verify the result

MCP is an integration choice. A direct API is enough for one fixed client.

Use MCP when multiple compatible hosts need live, reusable app capabilities.

MCP BUILD · six parts, one tool

YOUR EDIT: `mcp_server_sdk.py`

1. Register `set_swing(amount: int)`.
2. Describe its purpose and range.
3. Validate 0–75 in Python.
4. `call_daw: cmd=set_swing, amount=the validated input`.
5. Return the acknowledgement.
6. Read `get_state` to verify.

SERVER

Reuse the supplied FastMCP instance.

TOOL CONTRACT

Decorator + name + type + description.

VALIDATION + ADAPTER

Check the range; send the app command.

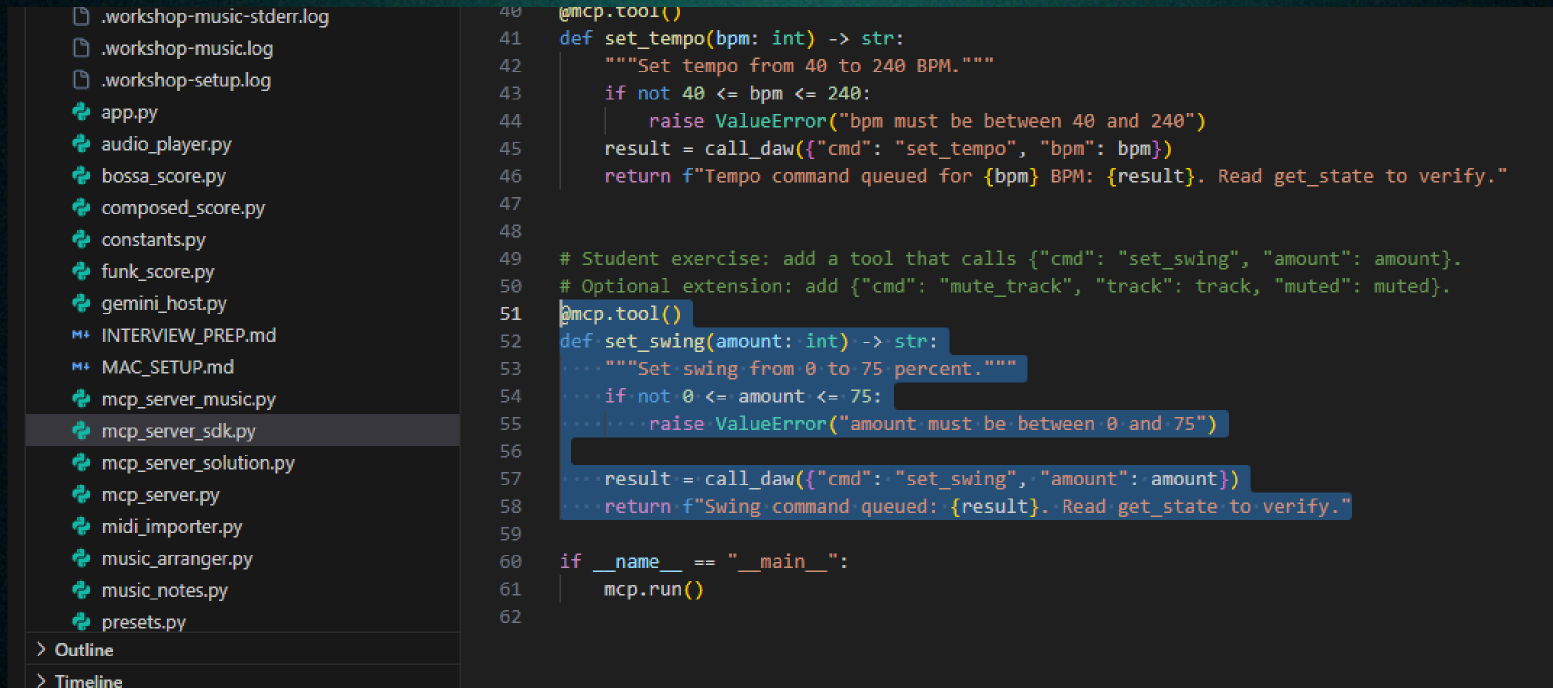
RESULT + TRANSPORT

Verify queued work. Keep stdio startup.

FastMCP handles the order channel. `app.py` does the work. `get_state` checks what actually changed.

MCP BUILD · edit mcp_server_sdk.py

1 Explorer → MCPJAM 2 mcp_server_sdk.py 3 Add below “Student exercise”



```
40 @mcp.tool()
41 def set_tempo(bpm: int) -> str:
42     """Set tempo from 40 to 240 BPM."""
43     if not 40 <= bpm <= 240:
44         raise ValueError("bpm must be between 40 and 240")
45     result = call_daw({"cmd": "set_tempo", "bpm": bpm})
46     return f"Tempo command queued for {bpm} BPM: {result}. Read get_state to verify."
47
48
49 # Student exercise: add a tool that calls {"cmd": "set_swing", "amount": amount}.
50 # Optional extension: add {"cmd": "mute_track", "track": track, "muted": muted}.
51 @mcp.tool()
52 def set_swing(amount: int) -> str:
53     """Set swing from 0 to 75 percent."""
54     if not 0 <= amount <= 75:
55         raise ValueError("amount must be between 0 and 75")
56
57     result = call_daw({"cmd": "set_swing", "amount": amount})
58     return f"Swing command queued: {result}. Read get_state to verify."
59
60 if __name__ == "__main__":
61     mcp.run()
62
```

Keep it above server startup. Save → reconnect → test → verify with get_state.

MCP TEST · discover, call, verify

- 1 Implement and save your tool.
- 2 Restart or reconnect to load the changes.
- 3 Confirm discovery and test explicit inputs.
- 4 Use `get_state` to verify the change.

2 / 3 · PRIZE QUESTION · Hands up!

Should `set_swing(100)` succeed?

A Yes B No



15 minutes.
Test and adapt.

25–40 min · build, reconnect, test, then explain to a partner

MCP TEST · evidence of success

Test	Expected evidence
Discovery	set_swing appears in available tools
Amount 35	Accepted and verified through get_state
Amount 0 and 75	Boundary values accepted
Amount 100	Tool error; Application state unchanged

When the valid call
AND the rejected call
both behave correctly.



3 / 3 · PRIZE QUESTION · Hands up!

Does “queued” prove the music changed?

A Yes B No

Absolute verification.

The security risks behind a tool call

UNAUTHORIZED ACCESS

Tools or records exposed to a caller who should not have access.

EXCESSIVE PERMISSIONS

A tool or backend account can do more than the task requires.

INDIRECT INJECTION

Malicious instructions hidden in documents or tool results.

“Just give every tool admin access.”



Range validation checks the value.
Authorization checks whether the caller may perform the action.

Tool poisoning and rug pulls

TOOL POISONING

Malicious instructions in a tool description try to manipulate the model or its workflow.

RUG PULLS

A previously trusted tool changes its description or behavior after approval.

Review what you connect—and review what changes.



Security controls for MCP tools

- ✓ Check access: identity, token audience, and permission for the action.
- ↓ Limit permissions: tools and backend accounts get only what they need.
- ! Treat external content as untrusted: documents and tool results can contain malicious instructions.
- ↻ Review code and updates: server sources, dependencies, and tool descriptions.

Advanced capabilities

Structured results, images, and audio where supported.

Local stdio or remote Streamable HTTP + authorization.

Workflows using tools from multiple servers.

Timeouts, outcome verification, and careful retries.

The host can retrieve from one server, call another, and save a result through a third. The host coordinates.

Enterprise: identity + production rules

IDENTITY PROVIDER

Establishes identity
Example: Entra ID

Identity / token issuance
to the client

Tool request →

OPTIONAL GATEWAY

Traffic policies and
monitoring

MCP →

MCP SERVER

Checks access to tools and
data

Application call →

BACKEND

Enforces Application
business rules

Production: per-tool authorization · audit logs · safe retries · verified outcomes.

Where can I use this at the hackathon?

ONE USEFUL ACTION

Search inventory or create a draft ticket

ONE STATE CHECK

Read the item / ticket and verify the result

ONE CLEAR BOUNDARY

Validate inputs; restrict access; approve risky writes



Exit check: name your tool, input, permission, and the evidence that proves it worked.